==========

RISE — AI ALARM CLOCK

MASTER BLUEPRINT: PART 13 OF 13 (REWRITTEN — CHANGESET)

Alarm Stakes (legally gated) · Bedtime Alarm · Sleep

Frequencies · CamPay + PawaPay (unaffected by the gating)

==========

GOVERNING DOCUMENT: RISE_DECISIONS.md Section 5,

RISE_COMPLIANCE_AUDIT.md Section 2.

SCOPE CORRECTION UP FRONT: only System 1 (Alarm Stakes) carries

legal exposure. Systems 2–4 (Bedtime Alarm, Sleep Frequencies,

CamPay/PawaPay) are legitimate, unrelated infrastructure bundled

into the same Part and need NO gating — verified by reading all

four systems' code, not assumed from the Part header.

A REAL TYPEID BUG FOUND HERE — AND ONE I INTRODUCED MYSELF: the

original Part 13 header lists "alarm_stake.dart → typeId 16,

charity_partner.dart → typeId 17, chronotype.dart → typeId 18," as

if it's one typeId per file. The actual code inside those files

declares FIVE Hive types, not three:

16 = StakeStatus (enum, in alarm_stake.dart)

17 = AlarmStake (class, in alarm_stake.dart)

18 = CharityPartner (class, in charity_partner.dart)

19 = ChronotypeType (enum, in chronotype.dart)

20 = Chronotype (class, in chronotype.dart)

The Part 12 rewrite trusted this Part's header summary instead of

its actual code and registered the wrong three adapters under the

wrong names (including an invented "ChronotypeProfile" that doesn't

exist — the real class is just `Chronotype`). That's fixed at the

bottom of this document. Worth being direct about: this is exactly

the mistake this whole project exists to catch, and I made it once

before catching it here.

A GENUINE INTEGRATION GAP, DOCUMENTED BY THE ORIGINAL BLUEPRINT

ITSELF: this Part's own integration notes say to add an "Add a

stake" tile to AlarmSetupScreen (Part 8). Part 8 was checked in the

Part 10B rewrite pass and confirmed to have zero stakes-related

code — that check was correct at the time; this integration point

was simply never built. Fixed below, gated the same way as

everything else in this Part.

A CAVEAT THIS REWRITE CANNOT CLOSE:

`initiateAlarmStake.js`**,**

`resolveAlarmStake.js`**, and** `cancelAlarmStake.js` **are server-side**

Cloud Functions, deployed separately, not part of this markdown

blueprint's content. Gating the Flutter client is necessary but NOT

sufficient — a client-side flag stops the app's UI from reaching

these functions, but doesn't stop a direct API call to them. Those

Cloud Functions need their OWN server-side check against the same

"stakes enabled" flag before processing any real payment. Flagging

this prominently since it's outside what a markdown blueprint rewrite

can fix, but it's a real gap if left unaddressed at deploy time.

FILES IN THIS PART:

1. alarm_stake.dart → CHANGED (typeId doc fix only)

2. charity_partner.dart → CHANGED (typeId doc fix only)

3. chronotype.dart → CHANGED (typeId doc fix only)

4. alarm_stake_service.dart → CHANGED (gating, dispute flow)

5. bedtime_alarm_service.dart → NO CHANGE (verified, unrelated)

6. sleep_frequency_service.dart → NO CHANGE (verified, unrelated)

7. campay_service.dart → NO CHANGE (see note below)

8. pawapay_service.dart → NO CHANGE (see note below)

9. stake_setup_screen.dart → CHANGED (gate check at entry)

10. bedtime_screen.dart → NO CHANGE (verified, unrelated)

11. payment_service_patch.dart → NO CHANGE (see note below)

NEW: lib/core/services/stakes_eligibility_service.dart

ALSO CHANGED: lib/features/alarm/alarm_setup_screen.dart (Part 8) —

adds the gated "Add a stake" tile Part 13 always assumed existed

==========

FILE 1: lib/core/models/alarm_stake.dart — CHANGED (doc only)

No code changes — `StakeStatus` **(16) and** `AlarmStake` **(17) are**

correct as originally written. Only the file-level header comment

is corrected to show both typeIds instead of implying one.

FILE 2: lib/core/models/charity_partner.dart — CHANGED (doc only)

No code changes needed — `CharityPartner` really is typeId 18, the

header list just mislabeled it as 17 by assuming one-per-file.

FILE 3: lib/core/models/chronotype.dart — CHANGED (doc only)

No code changes — `ChronotypeType` (19) and `Chronotype` (20) are

correct as written. Header corrected to show both.

NEW FILE: lib/core/services/stakes_eligibility_service.dart

```
import 'package:hive/hive.dart';
import '../constants/app_constants.dart';
import 'remote_config_service.dart';
import 'user_profile_service.dart';  // VERIFY this is the actual
                                     // service name/location for
                                     // user profile data — not
                                     // re-read as part of this pass

class StakesEligibilityService {
  static final StakesEligibilityService _i =
      StakesEligibilityService._internal();
  factory StakesEligibilityService() => _i;
  StakesEligibilityService._internal();

  // ─── HARD COMPILE-TIME GATE ──────────────────────────────────────
  // Flip to true only after gambling / money-transmitter / charitable-
  // solicitation legal opinions are back for every target jurisdiction
  // — see RISE_COMPLIANCE_AUDIT.md Section 2, DECISIONS.md Section 5.
  // This is a deliberate code change requiring a release, not a remote
  // config value — a server-side mistake or compromise should not be
  // able to silently turn real-money stakes on.
  static const bool _stakesCompiledGate = false;

  // Remote config can further RESTRICT (e.g. disable in one country
  // pending local review) but can never re-enable beyond what the
  // compiled constant above allows.
```

```
  Future<bool> isFeatureLive() async {
    if (!_stakesCompiledGate) return false;
    return RemoteConfigService()
        .getBool('stakes_enabled', defaultValue: true);
  }

  // 18+ required, no exceptions, regardless of jurisdiction.
  Future<bool> isOldEnough(String userId) async {
    final profile = await UserProfileService().getProfile(userId);
    return (profile?.age ?? 0) >= 18;
  }

  // Permanent, user-initiated, per-account self-exclusion.
  Future<bool> isSelfExcluded(String userId) async {
    final box = Hive.box<dynamic>(AppConstants.settingsBoxName);
    return box.get('stakes_self_excluded_$userId', defaultValue: false)
        as bool;
  }

  Future<void> setSelfExcluded(String userId, bool excluded) async {
    final box = Hive.box<dynamic>(AppConstants.settingsBoxName);
    await box.put('stakes_self_excluded_$userId', excluded);
    // The settings UI should treat turning this ON as a one-way door
    // (confirm dialog, no casual toggle back) even though the data
    // model itself allows reversal — friction is the intended design,
    // not a technical limitation.
  }

  // Single combined check — this is what every caller should use.
  Future<bool> canUseStakes(String userId) async {
    if (!await isFeatureLive())        return false;
    if (!await isOldEnough(userId))    return false;
    if (await isSelfExcluded(userId))  return false;
    return true;
  }
}
```

FILE 4: lib/core/services/alarm_stake_service.dart — CHANGED

Two additions: the eligibility check at the top of initiateStake(),

and a dispute-filing method. Everything else — seedCharities(),

getCharitiesForCountry(), maxStakeCentsForTier() (the tier-based cap

already existed and needed no change), formatStakeAmount(),

_listenToStakeUpdates(), _onStakeStatusChanged(),

getActiveStakeForAlarm(), getStakeHistory() — is UNCHANGED.

```
import '../services/stakes_eligibility_service.dart';  // NEW import

class AlarmStakeService {
  // ...unchanged singleton boilerplate, unchanged fields...

  final _eligibility = StakesEligibilityService();  // NEW

  // ─── Initiate a stake on an alarm — CHANGED: eligibility check added
  // as the very first thing this method does. Everything after the
  // check (building the AlarmStake, saving locally, calling the Cloud
  // Function, listening for updates) is UNCHANGED.
  Future<AlarmStake> initiateStake({
    required String alarmId,
    required int    amountCents,
    required String currency,
    required String recipientType,
    String?         charityId,
    String?         charityName,
    String?         friendUserId,
    String?         friendName,
    required String mobilePhone,
    required String paymentGateway,
  }) async {
    final prefs  = await SharedPreferences.getInstance();
    final userId = prefs.getString('userId') ?? '';

    // NEW — the entire reason for this rewrite:
    if (!await _eligibility.canUseStakes(userId)) {
      throw StakesUnavailableException();
    }
```

```
    // Everything below this line is UNCHANGED from the original.
    final stake = AlarmStake(
      id:             _uuid.v4(),
      // ...rest of construction and Cloud Function call, unchanged...
    );
    // ...
    return stake;
  }

  // ─── NEW: dispute filing ──────────────────────────────────────────────
  // Does NOT attempt to resolve anything client-side — a real-money
  // dispute needs human review, not an automated reversal. Routes to
  // the admin tooling (Section 6) via a support-ticket Cloud Function.
  // NOTE: `fileStakeDispute` is a NEW Cloud Function this rewrite
  // specifies but cannot implement (server-side, not part of this
  // markdown blueprint) — needed alongside the existing three.
  Future<void> fileDispute(String stakeId, String userExplanation) async {
    await _functions.httpsCallable('fileStakeDispute').call({
      'stakeId':     stakeId,
      'explanation': userExplanation,
    });
  }

  // seedCharities(), getCharitiesForCountry(), maxStakeCentsForTier(),
  // formatStakeAmount(), _listenToStakeUpdates(),
  // _onStakeStatusChanged(), getActiveStakeForAlarm(), cancelStake(),
  // getStakeHistory() — ALL UNCHANGED.
  //
  // SEPARATE, PRE-EXISTING ISSUE FOUND (not part of this rewrite's
  // scope, flagging for later): cancelStake()'s comment promises stakes
  // can't be cancelled "if < 2 hours before the alarm," but the actual
  // code only checks `status == held`, not the time. Not a legal/safety
  // issue, just a business-logic gap — noting it rather than fixing it
  // here to keep this changeset focused on the legal gating.
}

// NEW exception type:
class StakesUnavailableException implements Exception {
  final String message;
  const StakesUnavailableException([
    this.message = 'Alarm Stakes isn\'t available right now.',
  ]);
}
```

FILE 5: lib/core/services/bedtime_alarm_service.dart

NO CHANGE. Verified — calculates bedtime from wake time, sleep

goal, chronotype, and sleep debt. No money involved, no legal

exposure, no stakes references.

FILE 6: lib/core/services/sleep_frequency_service.dart

NO CHANGE. Verified — binaural beats / isochronic tone generation.

Unrelated to stakes.

FILE 7: lib/core/services/campay_service.dart

NO CODE CHANGE. This file has one stakes-specific method,

`initiateStakePayment()`**, alongside its general subscription-payment**

methods. It doesn't need its own gate — it's a low-level payment

adapter that only gets called BY `AlarmStakeService.initiateStake()`**,**

which is now gated. Once that entry point is blocked, this method is

simply unreachable. Gating belongs at the orchestration layer, not

duplicated into every leaf method underneath it. General CamPay

subscription-payment functionality is unaffected and unchanged.

FILE 8: lib/core/services/pawapay_service.dart

NO CODE CHANGE. Same reasoning as FILE 7.

FILE 9: lib/features/alarm/stake_setup_screen.dart — CHANGED

Adds an eligibility check when the screen opens. If ineligible,

shows an explanation instead of the stake-setup flow. Everything

else (amount picker, recipient selection, charity/friend picker,

phone entry, confirmation) is UNCHANGED.

```
import '../../core/services/stakes_eligibility_service.dart';  // NEW

class _StakeSetupScreenState extends ConsumerState<StakeSetupScreen> {
  final _svc         = AlarmStakeService();
  final _eligibility = StakesEligibilityService();  // NEW

  bool? _isEligible;  // NEW — null while checking, then true/false

  @override
```

```
  void initState() {
    super.initState();
    _checkEligibility();  // NEW
  }

  Future<void> _checkEligibility() async {
    final prefs  = await SharedPreferences.getInstance();
    final userId = prefs.getString('userId') ?? '';
    final result = await _eligibility.canUseStakes(userId);
    if (mounted) setState(() => _isEligible = result);
  }

  // build() — CHANGED: gate the whole screen behind the check.
  @override
  Widget build(BuildContext context) {
    if (_isEligible == null) {
      return const Scaffold(
        backgroundColor: Color(0xFF0A0A0F),
        body: Center(child: CircularProgressIndicator()),
      );
    }

    if (_isEligible == false) {
      return Scaffold(
        backgroundColor: const Color(0xFF0A0A0F),
        appBar: AppBar(
          backgroundColor: Colors.transparent,
          leading: const BackButton(color: Colors.white54),
        ),
        body: const Center(
          child: Padding(
            padding: EdgeInsets.all(24),
            child: Text(
              'Alarm Stakes isn\'t available for your account right
now.',
              textAlign: TextAlign.center,
              style: TextStyle(color: Colors.white70, fontSize: 15),
            ),
          ),
        ),
      );
    }

    // Everything from here down (the original build() body — intro
    // text, amount slider/presets, recipient picker, etc.) is
    // UNCHANGED.
  }

  // All other methods unchanged.
}
```

FILE 10: lib/features/alarm/bedtime_screen.dart

NO CHANGE. Verified — unrelated to stakes.

FILE 11: lib/core/services/payment_service_patch.dart

NO CODE CHANGE. Same reasoning as FILES 7/8 — its documented

`initiateStakePayment()` **routing addition is only reachable through**

the now-gated `AlarmStakeService`**, so it's unreachable, not**

rewritten. Its CamPay/PawaPay routing logic for general subscription

payments is unaffected.

ALSO CHANGED: lib/features/alarm/alarm_setup_screen.dart (Part 8)

The original Part 13's own integration notes documented an "Add a

stake" tile that was supposed to be added to this screen — never

actually built (confirmed clean in the Part 10B-rewrite pass, which

checked for stakes code that didn't yet exist). Added here, gated

the same way as the screen it opens.

```
import '../../core/services/stakes_eligibility_service.dart';  // NEW

// Inside AlarmSetupScreen's build method, among the other setup tiles
// (sound, volume, game, biometric, buddy, barcode game, nap mode) —
// NEW:
FutureBuilder<bool>(
  future: StakesEligibilityService().isFeatureLive(),
  builder: (context, snapshot) {
    if (snapshot.data != true) return const SizedBox.shrink();
    return ListTile(
      leading: const Icon(Icons.savings_outlined, color:
Colors.white54),
      title: const Text('Add a stake', style: TextStyle(color:
Colors.white)),
      subtitle: const Text('Put money on waking up',
          style: TextStyle(color: Colors.white38, fontSize: 12)),
      onTap: () => context.push(
        '/stake-setup?alarmId=${widget.editAlarmId}',
      ),
    );
  },
),
// The tile only checks isFeatureLive() (not full canUseStakes()) —
// age/self-exclusion get checked again at the setup screen itself
// (FILE 9 above), so showing the tile doesn't leak eligibility
// details, it just hides the entry point entirely while the feature
// is off. A corresponding '/stake-setup' GoRoute needs adding to
# app.dart alongside the other alarm routes.
```

PART 13 FILE INVENTORY (rewritten)

NEW: stakes_eligibility_service.dart
CHANGED: alarm_stake_service.dart, stake_setup_screen.dart, alarm_setup_screen.dart (Part 8), plus doc-only typeId fixes to alarm_stake.dart, charity_partner.dart, chronotype.dart
NO CHANGE (verified): bedtime_alarm_service.dart, sleep_frequency_service.dart, campay_service.dart, pawapay_service.dart, bedtime_screen.dart, payment_service_patch.dart

CORRECTIONS TO EARLIER DOCUMENTS IN THIS PROJECT

- DECISIONS.md Section 2's typeId note (16/17/18) is wrong — corrected set is 16 (StakeStatus), 17 (AlarmStake), 18 (CharityPartner), 19 (ChronotypeType), 20 (Chronotype).
- The Part 12 rewrite's adapter registration list used these wrong numbers and an invented class name (`ChronotypeProfile`, which doesn't exist — it's just `Chronotype`). Corrected alongside this document.
- DECISIONS.md Section 5 can now note the client-side gating is code-complete; still fully blocked on the actual legal opinions before `_stakesCompiledGate` flips to true.

STILL OPEN (not this rewrite's to close)

- Server-side enforcement in the three (soon four, with `fileStakeDispute`) Cloud Functions — client gating alone isn't sufficient, per the caveat at the top of this document.
- `UserProfileService` / `age` field name in the eligibility service above is written to the expected shape but not verified against Part 2's actual user_profile.dart in this pass.
- The pre-existing cancelStake() 2-hour-window comment/code mismatch, noted but not fixed (out of scope — not a legal issue).

PART 13 IS THE LAST NUMBERED PART

Per DECISIONS.md Section 9, what's left is new work with no existing Part to rewrite: agentic core (3.1), voice layer (3.3), Matter as a 4th smart-home provider (3.4, now correctly framed as additive), the on-device AI split (3.5), and the admin dashboard (Section 6).